Agentisches Programmieren

Joern Ploennigs · AI4SC



> There's a new kind of coding I call ‚vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists.
>
> — Andrej Karpathy, Beitrag auf X, 2025 [1]

Fehlerfrei ausgeführt heißt nicht richtig gerechnet

Prompt: „Berechne die Biegespannung eines Rechteckträgers."

```
b, h = 200, 400     # Breite, Höhe in mm
M = 150             # Moment in kNm
W = b * h**2 / 6    # Widerstandsmoment in mm³
sigma = M / W       # Spannung in N/mm²
print(f"Ergebnis: {sigma:.1e}")
```

```
Ergebnis: 2.8e-05
```

```
Ergebnis: 2,8·10⁻⁵
```

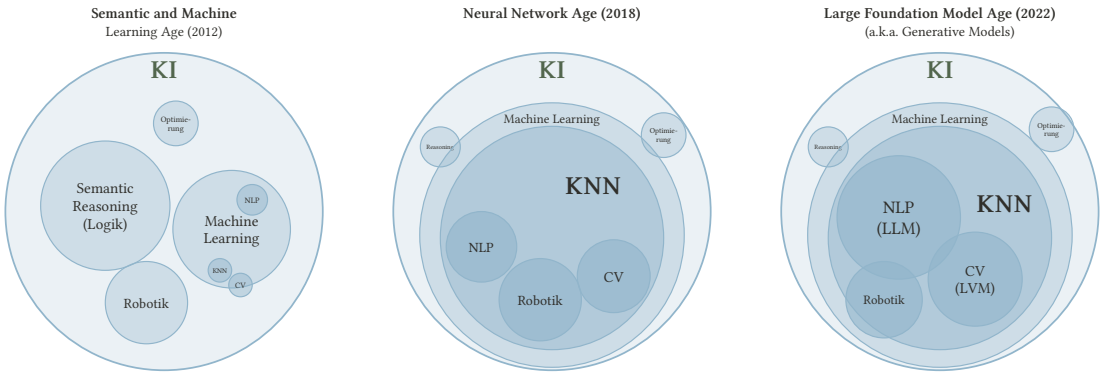
Was sehen Sie, wenn Sie den Code lesen?

- Kein Fehler, das Programm läuft durch
- Moment in kNm, Widerstandsmoment in mm³
- Es fehlt der Faktor 10^6 für die Einheiten
- Richtig: 28,125 N/mm², nicht 0,000028

Warum macht ein so leistungsfähiges System so einen Fehler?

KI-Assistenten können Code erzeugen, der ohne Fehlermeldung läuft und trotzdem falsch rechnet. Im Beispiel wird ein Moment in kNm durch ein Widerstandsmoment in mm³ geteilt; die Einheiten passen nicht zusammen, und die Spannung fällt um den Faktor 10^6 zu klein aus. Python meldet dabei nichts, denn für den Rechner ist jede Zahl gültig. Wer den Code liest, erkennt das sofort; wer ihn nur ausführt, nicht. Dieses Beispiel begleitet uns durch die ganze Sitzung. Zunächst fragen wir, wie ein Sprachmodell zu einem solchen Ergebnis kommt.

Gebiete der Künstlichen Intelligenz

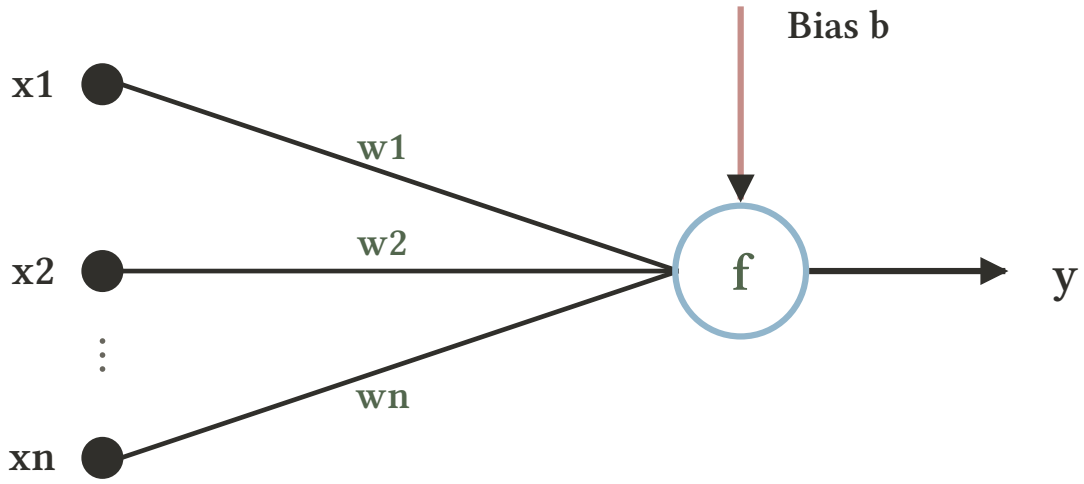


- KNN – Künstliche Neuronale Netzwerke
- CV – Computer Vision (Bildverarbeitung) und Large Vision Models (LVM)
- NLP – Natural Language Processing (Sprachverarbeitung) und Large Language Models (LLM)

Die Gebiete der Künstlichen Intelligenz haben sich in drei Zeitaltern verschoben. Im Semantic and Machine Learning Age (2012) stehen Semantic Reasoning (Logik), Machine Learning, Optimierung und Robotik nebeneinander. Im Neural Network Age (2018) wird Machine Learning zum dominierenden Gebiet, und Künstliche Neuronale Netzwerke (KNN) umfassen NLP, Robotik und Computer Vision (CV). Im Large Foundation Model Age (2022), auch Generative Models genannt, prägen Large Language Models (LLM) für die Sprachverarbeitung (NLP) und Large Vision Models (LVM) für die Bildverarbeitung (CV) die Neuronalen Netzwerke.

KNN – Künstliche Neuronale Netzwerke (1943)

- Mathematische Nachbildung der Neuronen im menschlichen Gehirn
- Ein Neuronales Netzwerk setzt sich aus (sehr) vielen Neuronen zusammen
- Jedes Neuron ist meist einfach aufgebaut
  - Eingänge $x_1, x_2, ..., x_n$ die multipliziert werden mit den
  - Gewichten $w_1, w_2, ..., w_n$ und dazu wird ein
  - Bias b addiert
- Der Ausgang y ist die Summe davon, gefiltert mit einer Aktivierungsfunktion $f(\cdot)$ (Wir gewichten also die Eingänge nach „Wichtigkeit“)
- Die Aufgabe beim Lernen ist die Gewichte $w_1, w_2, ..., w_n$ und das Bias b zu bestimmen (Wir lernen welche Eingänge wichtig sind)
- **Der Trick:** Neuronale Netzwerke bestehen aus einfachen Bausteinen (⟶ einfach hoch-parallel zu berechnen)



$$y = f\left(b + \sum_{i=1}^{n} w_i x_i\right)$$

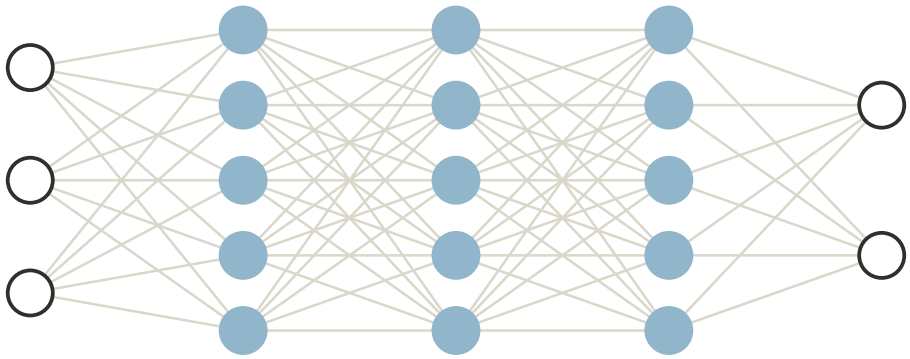
Um Fehler wie den obigen vorherzusehen, müssen wir verstehen, was ein Large Language Model (LLM) tatsächlich berechnet und was es nicht berechnet. Danach ordnen wir die gängigen Werkzeuge entlang ihrer Autonomie ein.

Künstliche neuronale Netzwerke (KNN, 1943) sind eine mathematische Nachbildung der Neuronen im menschlichen Gehirn. Ein Netzwerk setzt sich aus sehr vielen Neuronen zusammen, und jedes Neuron ist meist einfach aufgebaut: Die Eingänge $x_1, ..., x_n$ werden mit den Gewichten $w_1, ..., w_n$ multipliziert, und dazu wird ein Bias b addiert. Der Ausgang y ist die Summe davon, gefiltert mit einer Aktivierungsfunktion $f(\cdot)$. Wir gewichten also die Eingänge nach Wichtigkeit. Die Aufgabe beim Lernen ist, die Gewichte und das Bias zu bestimmen, wir lernen also, welche Eingänge wichtig sind. Der Trick: Neuronale Netzwerke bestehen aus einfachen Bausteinen, die gut zu rechnen sind.

DNN – Tiefe Neuronale Netzwerke (seit 2011 skalierbar)

- Tiefe Neuronale Netzwerke enthalten sehr viele (3-150) tiefe Schichten von Neuronen
- Mit jeder Schicht steigt die Anzahl der Parameter multiplikativ und man erhält tausende Parameter
- Ab 2011 führten zwei Entwicklungen zur Berechenbarkeit
  - Parallelisierung der Berechnung in Graphikkarten mit tausenden von Recheneinheiten
  - Bessere Algorithmen, um die sehr große Anzahl an Parametern schnell zu bestimmen
- **Der Trick:** Mit den tieferen Schichten kann das Neuronale Netzwerk Wissen besser
  - **Abstrahieren** – da wir Eingänge immer mehr zusammen führen

- **Verknüpfen** – Mit jeder Schicht verknüpfen wir das abstrahierte Wissen besser und betonen das Wichtige



Eingabeschicht **Verdeckte Schichten** **Ausgabeschicht**

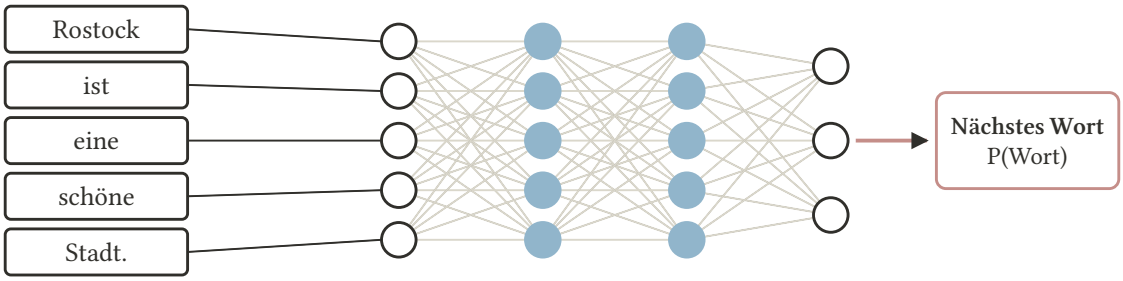
Tiefe Neuronale Netzwerke (DNN) enthalten sehr viele (3 bis 100) tiefe Schichten von Neuronen. Mit jeder Schicht steigt die Anzahl der Parameter multiplikativ, und man erhält tausende von Parametern. Ab 2011 führten zwei Entwicklungen zur Berechenbarkeit: die Parallelisierung der Berechnung in Grafikkarten mit tausenden von Recheneinheiten und bessere Algorithmen, um die sehr große Anzahl an Parametern schnell zu bestimmen. Der Trick: Mit den tieferen Schichten kann das Netzwerk Wissen besser abstrahieren, da wir Eingänge immer mehr zusammenführen, und verknüpfen, da wir mit jeder Schicht das abstrahierte Wissen besser verknüpfen und das Wichtige betonen.

Große Sprachmodelle (Large Language Models LLM) (seit 2017)

- Erhöhen die Anzahl der Parameter auf mehrere Milliarden
- Die Netzwerke zerlegen Texte in Worte und lernen mit welcher Wahrscheinlichkeit diese Worte im Text aufeinander folgen
- Mit der Entdeckung von Transformer Netzwerken (2017) konnte man diese Textzusammenhänge lernen
- **Der Trick:** dass das Netzwerk lernt
  - **Abstrahieren** – was sind die wichtigen Worte in einem Text
  - **Verknüpfen** – welche Worte sind mit diesen wichtigen Worten über Sätze verknüpft
- Wenn wir eine Anfrage an ChatGPT stellen, so wird das umgekehrt und ChatGPT generiert die wahrscheinliche Wortfolge für meine Anfrage

[2]

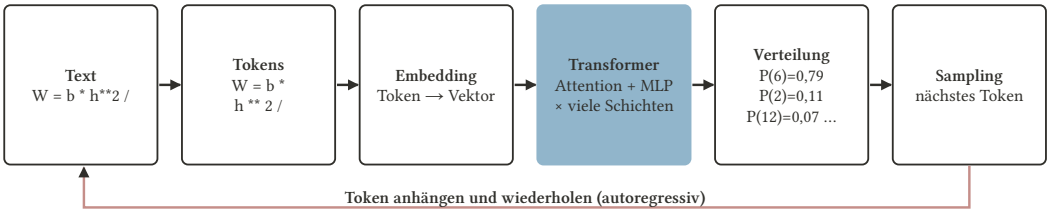
Vaswani et al.: Attention Is All You Need, NeurIPS, 2017.



Eingabe: Wörter als Zahlen

Große Sprachmodelle (Large Language Models, LLM, seit 2017) erhöhen die Anzahl der Parameter auf mehrere Milliarden. Die Netzwerke zerlegen Texte in Worte und lernen, mit welcher Wahrscheinlichkeit diese Worte im Text aufeinander folgen. Mit der Entdeckung der Transformer-Netzwerke [2] konnte man diese Textzusammenhänge lernen. Der Trick besteht darin, dass das Netzwerk lernt zu abstrahieren, welche Worte in einem Text wichtig sind, und zu verknüpfen, welche Worte mit diesen wichtigen Worten über Sätze verknüpft sind. Wenn wir eine Anfrage an ChatGPT stellen, wird das umgekehrt, und ChatGPT generiert die wahrscheinliche Wortfolge für die Anfrage.

Ein Sprachmodell bildet Token-Folgen auf Wahrscheinlichkeiten ab



Attention: Jedes Token gewichtet alle vorherigen Tokens nach Relevanz.

Ein LLM verarbeitet keinen Text, sondern Token, also Wortteile oder Symbole eines festen Vokabulars. Jedes Token wird in einen Vektor eingebettet. Mehrere Dutzend Transformer-Schichten berechnen daraus Zwischendarstellungen; in jeder Schicht gewichtet der Attention-Mechanismus für jedes Token, welche vorherigen Tokens für die Fortsetzung relevant sind. Am Ende steht eine Wahrscheinlichkeitsverteilung über das gesamte Vokabular. Aus ihr wird ein Token gezogen, an die Eingabe angehängt, und der Vorgang wiederholt sich. Man nennt das autoregressive Generierung. Im Beispiel steht nach `W = b * h**2 /` mit hoher Wahrscheinlichkeit eine 6, weil diese Formel im Training oft vorkam. Die Wahrscheinlichkeiten im Bild sind illustrativ. Moderne Modelle haben Milliarden bis Billionen Parameter.

Das Training belohnt Plausibilität, nicht Korrektheit

Training

1. Pre-Training: Fortsetzung von Text und Code vorhersagen
2. Instruction-Tuning: Aufgaben und Antworten nachahmen
3. Feedback: Antworten bevorzugen, die Menschen besser bewerten

$$\mathcal{L}(\theta) = -\sum_{t} \log P_\theta(x_t \mid x_{<t})$$

Was dieses Signal nicht enthält:

- Keine Einheitenprüfung
- Kein Ergebnis einer Ausführung
- Keine Kenntnis Ihrer Anforderung

Das Modell verallgemeinert Muster. Es schlägt nichts nach und rechnet nichts nach.

Beim Pre-Training minimiert das Modell den Vorhersagefehler für das nächste Token über sehr große Text- und Code-Sammlungen. Die Verlustfunktion bewertet nur, wie gut die tatsächliche Fortsetzung vorhergesagt wurde. Instruction-Tuning und Feedback von Menschen [3] richten das Modell auf hilfreiche Antworten aus. In keinem dieser Schritte kommt ein Ergebnis einer Programmausführung oder eine Einheitenprüfung vor. Das Modell erzeugt deshalb Code, der zu seinen Trainingsmustern passt. Es ist kein Nachschlagewerk: Auch Code, der so nie in den Trainingsdaten stand, kann es neu zusammensetzen. Das erklärt sowohl seine Stärke als auch den Einheitenfehler im Beispiel.

Programm code garantieren Korrektheit, ein Sprachmodell nicht

Programmcode

- Eingabe: Daten
- Gleiche Eingabe, gleiche Ausgabe
- Fehler: Fehlermeldung mit Zeile

Sprachmodell

- Eingabe: Prompt
- Keine formale Garantie für die Ausgabe
- Fehler: trotzdem eine plausible Ausgabe

Wer garantiert die Korrektheit? Bei einem LLM sind Sie es.

Programmcode verhält sich deterministisch: Dieselbe Eingabe liefert dieselbe Ausgabe, und Syntax- oder Typfehler führen zu einer Fehlermeldung mit Zeilenangabe. Ob die Formel im Code richtig ist, garantiert das nicht, wie unser Beispiel mit dem Einheitenfehler zeigt. Das Verhalten ist aber reproduzierbar und prüfbar. Ein Sprachmodell antwortet immer, auch wenn der Prompt unklar ist oder die richtige Antwort nirgends gelernt wurde, und dieselbe Eingabe kann andere Ausgaben liefern. Die Prüfung liegt hier bei den Anwenderinnen und Anwendern. Verifikation ist deshalb Pflicht und kein optionaler Zusatzschritt.

Warum nicht?

Wie ein Ingenieur denkt

1. Übersetzt ein Problem in ein **mentales Modell**
2. Analysiert darin abstrakte Zusammenhänge (versteht **warum und wann** der Balken sich biegt)
3. Kann kausale Schlussfolgerungen **ableiten**
4. Kann prüfen, ob das Ergebnis physikalisch sinnvoll ist (**kann rechnen**)

Wie ein LLM „denkt"

1. Übersetzt ein Problem in ein **sprachliches Muster**
2. Hat **kein** mentales Modell abstrakter Zusammenhänge (weiß nur, dass „Balken" und „biegen" korrelieren)
3. Sagt **nur** statistisch wahrscheinliche Wörter voraus
4. Kann nicht prüfen, ob der Output sinnvoll ist (**kann nicht rechnen**)

[4]

Ploennigs et al.: *Building foundation models - potentials, challenges and research directions for using LLM and LVM in AEC*, **EC3**, 2025.

Ein Ingenieur übersetzt ein Problem in ein mentales Modell und analysiert darin abstrakte Zusammenhänge: Er versteht, warum und wann sich ein Balken biegt. Er kann kausale Schlussfolgerungen ableiten und prüfen, ob das Ergebnis physikalisch sinnvoll ist, er kann also rechnen. Ein LLM dagegen übersetzt ein Problem in ein sprachliches Muster. Es hat kein mentales Modell abstrakter Zusammenhänge und weiß nur, dass „Balken" und „biegen" korrelieren. Es sagt nur statistisch wahrscheinliche Wörter voraus und kann nicht prüfen, ob der Output sinnvoll ist, es kann also nicht rechnen [4].

Die Gefahren neuer Grundlagen KI-Modelle

Verstehen

- *Fehlendes räumliches und numerisches* Verständnis
- *Fehlendes logisches Erschließen* komplexer Zusammenhänge obwohl einfache funktionieren
- *Auslassen* statistisch irrelevanter aber fachlich wichtiger Details
- *Übersetzen* mit False-Friends, die in der Domäne andere Bedeutungen haben

Generieren

- *Halluzination* von falschen Inhalten
- *Fehlassistenz* durch fehlerhafte Ratschläge
- *Inkonsistenz* von generierten Inhalten (mehrere Sichten)
- *Bias* der Vorurteile/ Falschaussagen in den Trainingsdaten überbetont
- *Ineffizienz* durch generierte Texte, die keiner mehr liest
- *Fehloptimierungen* durch Lernen von Korrelationen in Trainingsdaten

Anpassen

- *Keine BIM-Unterstützung* zur Datenverarbeitung in KI-Modellen
- *Fehlende große Datensätze*
- *Unerklärliche* Black-Box-Modelle
- *Hohe Kosten* für regelmäßiges Training
- *Fehlendes Wissen* zu den Modellen

Die Gefahren neuer Grundlagen-KI-Modelle lassen sich nach den drei Fähigkeiten Verstehen, Generieren und Anpassen ordnen. Beim Verstehen fehlen räumliches und numerisches Verständnis sowie das logische Erschließen komplexer Zusammenhänge, obwohl einfache funktionieren. Statistisch irrelevante, aber fachlich wichtige Details werden ausgelassen, und beim Übersetzen entstehen False-Friends, die in der Domäne andere Bedeutungen haben. Beim Generieren treten Halluzinationen von falschen Inhalten und Fehlassistenz durch fehlerhafte Ratschläge auf. Generierte Inhalte können inkonsistent sein (mehrere Sichten), Vorurteile und Falschaussagen in den Trainingsdaten werden überbetont, generierte Texte werden von niemandem mehr gelesen (Ineffizienz), und durch das Lernen von Korrelationen in Trainingsdaten entstehen Fehloptimierungen. Beim Anpassen fehlt die BIM-Unterstützung zur Datenverarbeitung in KI-Modellen, es fehlen große Datensätze, die Black-Box-Modelle sind unerklärlich, das regelmäßige Training ist teuer, und das Wissen zu den Modellen fehlt.

Kann KI das allein? → Die Kompetenzfalle

Verstehen

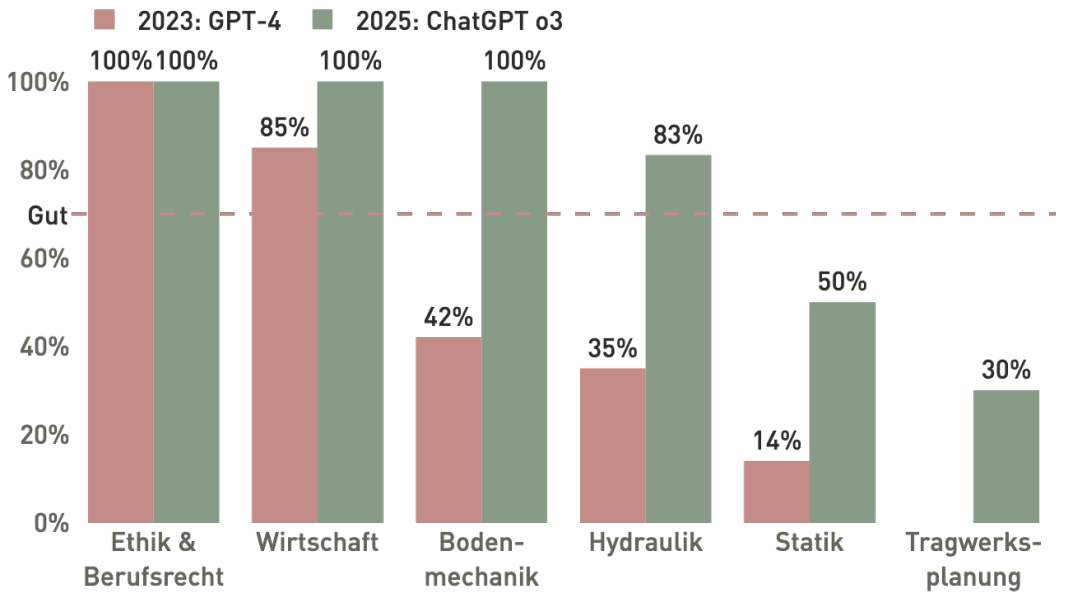
- Zusammenfassen, Allgemeinwissen zu Bauen und Architektur
- Kein numerisches oder räumliches Verständnis
- Kein Detailwissen zur Baudomäne und zu Projektinformationen

Generieren

- Übersetzen, Bilder erzeugen
- Halluziniert bei Unwissenheit
- Erzeugt keine wiederholbar zusammenhängenden Ergebnisse

Anpassen

- Wissen erweiterbar (RAG), Tuning von Modellen auf Spezialwissen
- Dennoch nicht an komplexe Ingenieurprobleme anpassbar



Daten: V. Pursnani, Y. Sermet, M. Kurt, und I. Demir [5], L. Oblitas, S. Adeeb, und C. Cruz-Noguez [6]. Statik und Tragwerke bleiben am schwächsten.

[5], [6]

Pursnani et al.: *Performance of ChatGPT on the US fundamentals of engineering exam*, **Computers and Education: AI**, 2023.

Oblitas, Adeeb & Cruz-Noguez: *Performance of AI Chatbots on the Fundamentals of Engineering Civil Exam*, **Discover Artificial Intelligence**, 2025.

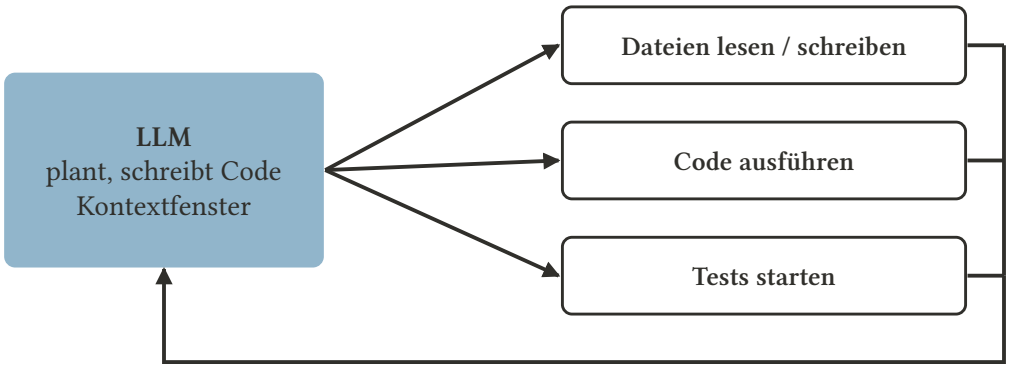
Kann KI Ingenieuraufgaben allein lösen? Das hängt davon ab, welche Fähigkeit wir betrachten. Beim Verstehen sind Zusammenfassen und Allgemeinwissen zu Bauen und Architektur Stärken, numerisches und räumliches Verständnis sowie Detailwissen zur Baudomäne fehlen. Beim Generieren gelingen Übersetzen und Bilder, bei Unwissenheit halluziniert das Modell und erzeugt keine wiederholbar zusammenhängenden Ergebnisse. Beim Anpassen lässt sich das Wissen über RAG erweitern und das Modell auf Spezialwissen abstimmen, an komplexe Ingenieurprobleme lässt es sich dennoch nicht anpassen. Die Aufgaben der US-amerikanischen Ingenieur-Grundlagenprüfung zeigen das: GPT-4 erreichte 2023 in der Statik 14 Prozent, ChatGPT o3 2025 immerhin 50 Prozent, in der Tragwerksplanung 30 Prozent. Statik und Tragwerke bleiben die schwächsten Gebiete [5], [6].

Agentenische Programmierung

Code-Vervollständigung wurde in acht Jahren zum Agenten

Die Technik hinter den heutigen Assistenten ist jung. Die Transformer-Architektur [2] ersetzte 2017 rekurrente Netze durch den Attention-Mechanismus. GPT-3 [7] zeigte 2020, dass ein ausreichend großes Modell Aufgaben aus wenigen Beispielen im Prompt lernt. Codex, das Modell hinter GitHub Copilot, wurde 2021 auf Quellcode feinjustiert [8]. Mit Instruction-Tuning und Feedback von Menschen [3] entstanden 2022 Chat-Assistenten. Seit 2025 verbreiten sich Coding-Agenten, die Dateien lesen, Code ausführen und Tests starten. Karpathy prägte für das ungeprüfte Übernehmen solcher Ergebnisse den Begriff vibe coding [1].

Ein Agent ist ein Sprachmodell in einer Schleife mit Werkzeugen



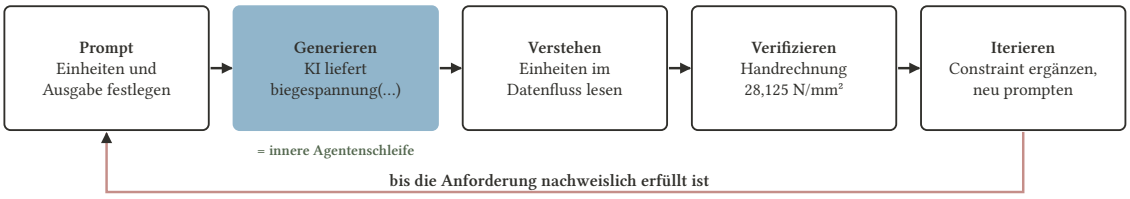
Der Agent schleift selbstständig.
Die Anforderung kennt nur der Mensch.

Autonomie der Werkzeuge:

- GitHub Copilot ergänzt Zeilen beim Tippen. Prüfen: jede Annahme per Tab
- Cursor bietet Chat und Edits im Editor. Prüfen: den Diff vor der Übernahme
- Claude Code plant, liest, führt aus und testet. Prüfen: das Ergebnis gegen die Anforderung

Ein Agent erweitert das Sprachmodell um eine Schleife: Das Modell plant einen Schritt, ruft ein Werkzeug auf, etwa zum Lesen einer Datei, zum Ausführen von Code oder zum Starten von Tests, und erhält das Ergebnis zurück in sein Kontextfenster. So arbeitet es mehrere Schritte selbstständig ab. Die drei Werkzeuge dieses Kurses unterscheiden sich vor allem im Grad der Autonomie. GitHub Copilot liefert Inline-Vorschläge im Tippfluss, was bequem ist, aber dazu verleitet, Code ohne Lesen zu akzeptieren. Cursor kombiniert Editor und Chat. Claude Code, das Werkzeug dieses Kurses, arbeitet im Terminal oder als VS-Code-Erweiterung mit Sicht auf das ganze Projekt. Je höher die Autonomie, desto wichtiger ist die Prüfung, denn der Agent kennt Ihre Anforderung nur aus dem Prompt.

Der agentische Zyklus legt eine äußere Schleife um den Agenten



Jeder Schritt braucht eine eigene Kompetenz: Anforderungen formulieren, Code lesen, Testen denken.

→ `01a_Ueberblick` (S01): Kursüberblick und Symbollegende

Weil der Agent nur so gut ist wie die Anforderung, die er erhält, brauchen Ingenieurinnen und Ingenieure eine eigene, äußere Schleife. Sie ist die Arbeitsmethode des ganzen Kurses.

Der Zyklus besteht aus fünf Schritten: Prompt, Generieren, Verstehen, Verifizieren und Iterieren. Nur der zweite Schritt gehört der KI, die übrigen vier gehören Ihnen. Am Beispiel heißt das: Sie legen Einheiten und Ausgabegröße fest, die KI liefert eine Funktion, Sie verfolgen die Einheiten im Datenfluss, prüfen das Ergebnis gegen die Handrechnung von 28,125 N/mm² und ergänzen bei Abweichung den fehlenden Constraint im Prompt. Der Zyklus wird wiederholt, bis die Anforderung nachweislich erfüllt ist. Er verlangt genau die Fähigkeiten, die dieser Kurs aufbaut: Python-Lesekompetenz, Anforderungsformulierung und Testdenken.

Ein Prompt ist eine Anforderungsspezifikation

1. **Rolle:** Tragwerksplanerin, Python-Entwickler
2. **Kontext:** b, h in mm, M in kNm
3. **Aufgabe:** `biegespannung(b_mm, h_mm, M_knm)`
4. **Constraint:** intern in N und mm, keine Bibliotheken
5. **Ausgabeformat:** Docstring mit Einheiten, Typ-Hints

> „Du bist Python-Entwickler für Tragwerksplanung.
> Eingaben: b und h in mm, M in kNm.
> Schreibe `biegespannung(b_mm, h_mm, M_knm) -> float`.
> Rechne intern in N und mm, Ergebnis in N/mm².
> Keine externen Bibliotheken.
> Mit Docstring, der die Einheiten nennt, und Typ-Hints.“

→ `04a_Anforderungen` (S04): Anforderungsspezifikation als Quelle eines guten Prompts

Ein guter Prompt hat fünf Elemente: Rolle, Kontext, Aufgabe, Constraint und Ausgabeformat. Der vage Prompt „Berechne die Biegespannung“ lässt alle offen: Einheiten, Funktionsname, Rückgabe, erlaubte Bibliotheken. Die KI muss raten, und sie rät plausibel. Der ausformulierte

Prompt legt diese Punkte fest und ist damit eine Anforderungsspezifikation im Sinne der Softwareentwicklung. Je präziser der Prompt, desto weniger Iterationen sind nötig. Die systematische Formulierung von Anforderungen vertiefen wir in Sitzung S04.

Der fehlende Constraint verursacht den Einheitenfehler

Vager Prompt:

```
def biegespannung_v1(b, h, M):
    W = b * h**2 / 6
    return M / W

print(f"{biegespannung_v1(200, 400, 150):.1e}")
```

```
2.8e-05
```

Läuft, aber 10^6-fach zu klein

Prompt mit Einheiten-Constraint:

```
def biegespannung(b_mm, h_mm, M_knm):
    """Biegespannung in N/mm²; M in kNm."""
    W_mm3 = b_mm * h_mm**2 / 6
    return M_knm * 1e6 / W_mm3

print(f"{biegespannung(200, 400, 150):.3f}")
```

```
28.125
```

Einheiten im Namen, Ergebnis stimmt

Beide Funktionen sind syntaktisch einwandfrei, und beide laufen ohne Fehlermeldung. Die erste entspricht dem Ergebnis des vagen Prompts: Sie rechnet mit gemischten Einheiten. Die zweite entsteht, wenn der Prompt die Einheiten festlegt: Die Parameternamen tragen die Einheiten, die Umrechnung steht explizit im Code, und der Docstring nennt die Einheit des Ergebnisses. Das ist kein Fehler des Modells, sondern ein unvollständiger Prompt. Die Ingenieurin oder der Ingenieur ist für die vollständige Spezifikation verantwortlich.

Lesen Sie zuerst den Vertrag der Funktion, dann die Implementierung

1. Signatur: Parameter, Einheiten, Rückgabetyp
2. Docstring: Was verspricht die Funktion?
3. Tests: Was ist das erwartete Verhalten?
4. Implementierung: Wie ist es umgesetzt?

Agentischer Ablauf

Wenn KI eine Funktion generiert:

1. Signatur lesen: Tragen die Namen die Einheiten?
2. Docstring prüfen: Stimmt die Einheit des Ergebnisses?
3. Testfall schreiben, bevor Sie die Implementierung lesen
4. Bei Abweichung: Prompt um den Constraint ergänzen

→ `08a_UnitTest` (S08): Tests als primäres Verifikationswerkzeug

Signatur und Docstring beschreiben das Was, die Implementierung das Wie. Wenn das Was nicht stimmt, ist das Wie irrelevant. Deshalb beginnt die Lesestrategie mit dem Vertrag der Funktion. In `biegespannung(b_mm, h_mm, M_knm)` stehen die Einheiten bereits in der Signatur; in `biegespannung_v1(b, h, M)` fehlen sie, und genau dort lag der Fehler. Die Reihenfolge, den Test vor der Implementierung zu lesen, ist eine Methode dieses Kurses und kein etablierter Branchenbegriff. Wer zuerst den Test formuliert, liest aus der Anforderungsperspektive statt aus der Implementierungsperspektive.

Ein Test mit Handrechnung findet den Fehler, bevor Sie lesen

```
erwartet = 28.125                    # Handrechnung in N/mm²
assert abs(biegespannung_v1(200, 400, 150) - erwartet) < 1e-3
```

```
AssertionError:
[31m-------------------------------------------------------------------------------[
[39m
[31mAssertionError[39m                            Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[13][39m[32m, line 2[39m
[32m      1[39m erwartet = [32m28.125[39m
[38;5;66;03m# Handrechnung in N/mm²[39;00m
[32m----> [39m[32m2[39m [38;5;28massert[39;00m
[38;5;28mabs[39m(biegespannung_v1([32m200[39m, [32m400[39m,
[32m150[39m) - erwartet) < [32m1e-3[39m

[31mAssertionError[39m:
```

```
assert abs(biegespannung(200, 400, 150) - erwartet) < 1e-3
print("Test bestanden")
```

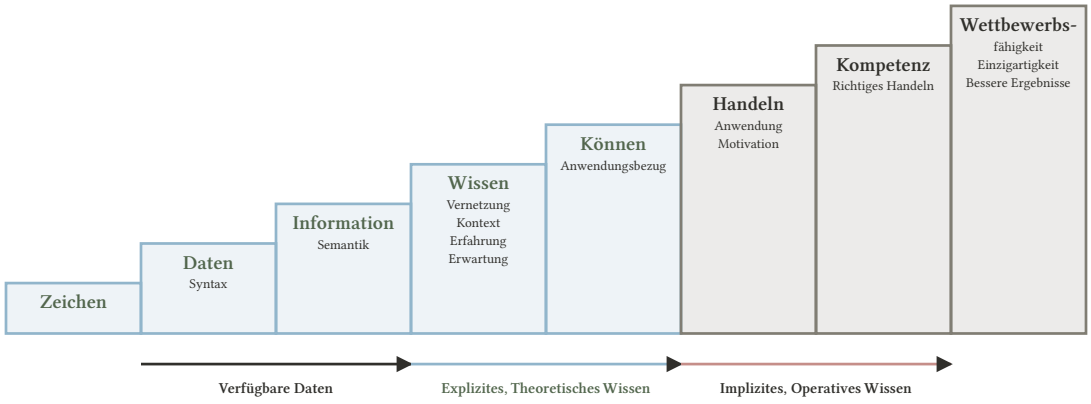
```
Test bestanden
```

Eigener Test: Was passiert bei h = 0? Welches Verhalten verlangt die Anforderung?

→ `08a_UnitTest` (S08): Unit-Tests und Test-first-Prompting

Der Test besteht aus einer einzigen Zeile: Die Handrechnung liefert 28,125 N/mm², und die Funktion muss diesen Wert bis auf eine Toleranz treffen. Gegen die erste Funktion schlägt der Test fehl, gegen die zweite besteht er. Der Test entsteht aus der Anforderung und nicht aus dem generierten Code, er kann deshalb nicht denselben Denkfehler enthalten. Bei h gleich null bricht die Funktion mit einer Division durch null ab. Ob das gewollt ist oder ob die Anforderung eine eigene Fehlermeldung verlangt, entscheidet die Anforderung, nicht die KI.

Wissenstreppe

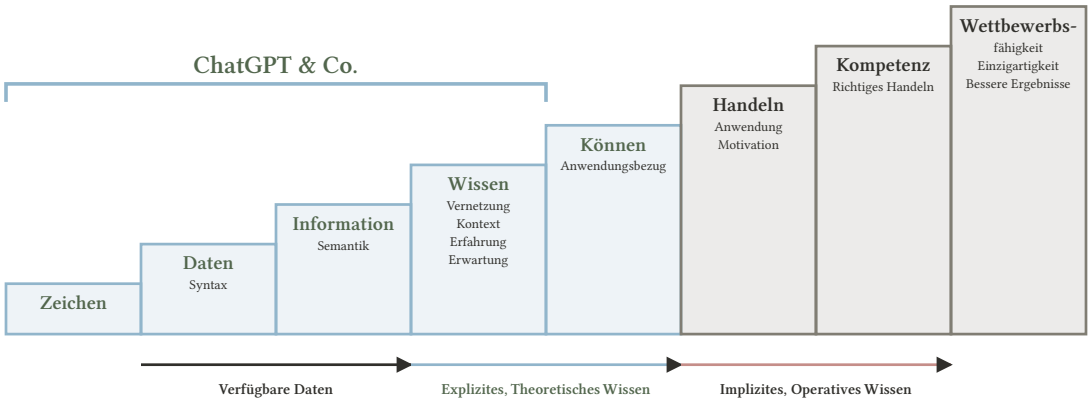


[9]

North, K. (2011): Wissensorientierte Unternehmensführung. Wertschöpfung durch Wissen.

Die Wissenstreppe nach North [9] erweitert die Wissenspyramide (Sitzung S03): Auf Zeichen folgen Daten (Syntax), Information (Semantik), Wissen (Vernetzung, Kontext, Erfahrung, Erwartung), Können (Anwendungsbezug), Handeln (Anwendung, Motivation), Kompetenz (richtiges Handeln) und Wettbewerbsfähigkeit (Einzigartigkeit, bessere Ergebnisse). Die unteren Stufen sind verfügbare Daten, die mittleren explizites, theoretisches Wissen und die oberen implizites, operatives Wissen.

Wissenstreppe und ChatGPT



[9]

North, K. (2011): Wissensorientierte Unternehmensführung. Wertschöpfung durch Wissen.

ChatGPT und vergleichbare Modelle decken die Stufen von den Zeichen bis zum Wissen ab. Das implizite, operative Wissen, also Handeln, Kompetenz und Wettbewerbsfähigkeit, liegt nicht in den Modellen.

KI schreibt den Code, Sie verantworten ihn

Auftraggeber

- Anforderung präzise formulieren
- Im Beispiel: Einheiten fehlten im Prompt

Leser

- Code und Annahmen verstehen
- Im Beispiel: kNm durch mm³ im Datenfluss

Verifizierer

- Ergebnis gegen die Anforderung prüfen
- Im Beispiel: Handrechnung ergibt 28,1

Die KI ist die Junior-Entwicklerin, Sie sind der Senior, der den Code abnimmt.

Beim Einsatz von KI-Werkzeugen übernehmen Ingenieurinnen und Ingenieure drei Rollen. Als Auftraggeber formulieren sie die Anforderung so präzise, dass die KI passenden Code erzeugen kann. Als Leser verstehen sie den generierten Code mit seinen Annahmen und Grenzen. Als Verifizierer prüfen sie, ob das Ergebnis die Anforderung erfüllt. Die Metapher von Junior und Senior ist bewusst gewählt: Eine gute Senior-Entwicklerin oder ein guter Senior-Entwickler liest den Code gründlich, stellt Fragen, fordert Änderungen und gibt erst dann das Okay. Die Verant-

wortung für den Code in Produktion bleibt bei den Menschen; KI-Code entbindet nicht von der Haftung.

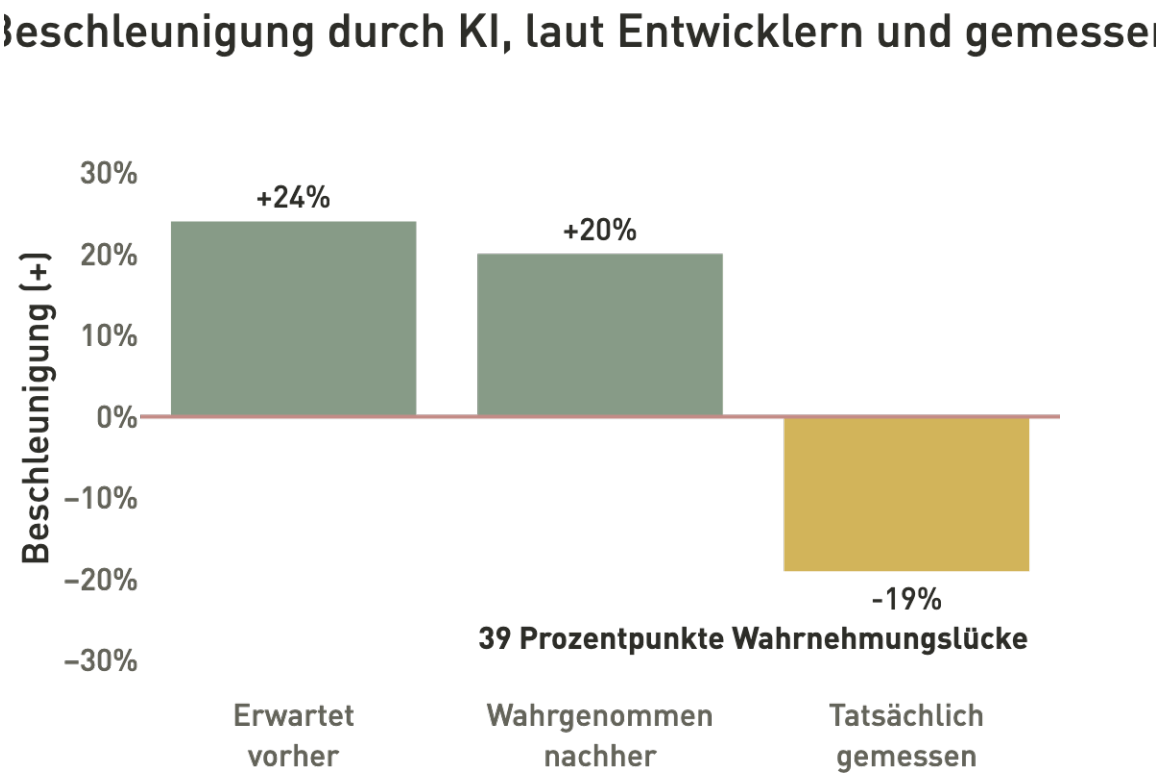
Vier Fehlerarten treten bei KI-Code immer wieder auf

Vier Fehlerarten sollten Sie beim Lesen gezielt suchen: erfundene Funktionen oder Bibliotheken, stillschweigend falsche Einheiten, vergessene Randfälle wie leere Listen oder Division durch null und Code, der ohne Fehlermeldung das Falsche berechnet. Hinzu kommen Fragen des Datenschutzes und der Lizenzen, wenn Projektdaten oder fremder Code an KI-Werkzeuge gelangen.

Dann prüft eben der Mensch? → Die Kapazitätsfalle

Das Prüfen von KI-Output ist ermüdend („AI Fatigue“) und nicht trivial, Beispiel Softwareentwicklung:

- KI-Output skaliert schneller als menschliche Review-Kapazität (4× mehr Pull Requests auf GitHub [10])
- Die gefühlte Produktivität steigt [11]
- Die gemessene Produktivität sinkt [11]



„Entwickler erwarteten, mit KI 24 % schneller zu sein, und selbst nachdem sie die Verlangsamung erlebt hatten, glaubten sie noch, KI habe sie um 20 % beschleunigt.“

— METR [11]

[10], [11]

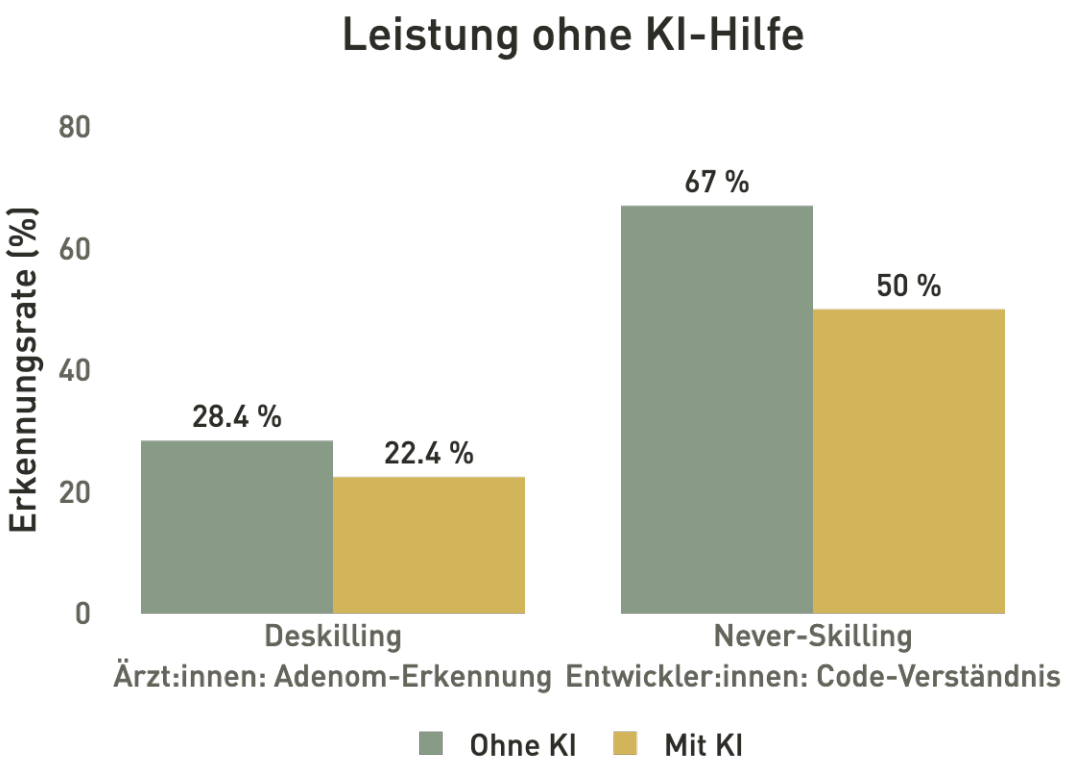
METR: *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*, 2025.

CodeRabbit: *GitHub Gives Maintainers a Throttle for the AI Pull Request*, 2026.

Auch wenn KI nicht allein arbeiten kann, bleibt das Prüfen nicht automatisch beim Menschen gut aufgehoben. Prüfen ist ermüdend („AI Fatigue“) und nicht trivial. In der Softwareentwicklung wächst der KI-Output schneller als die Review-Kapazität: Auf GitHub gibt es viermal mehr Pull Requests [10]. Die METR-Studie ließ 16 erfahrene Open-Source-Entwickler an 246 echten Issues arbeiten. Sie erwarteten, mit KI 24 Prozent schneller zu sein, und glaubten auch nach der Studie noch an 20 Prozent Beschleunigung. Tatsächlich waren sie 19 Prozent langsamer. Gefühlte und gemessene Produktivität liegen damit 39 Prozentpunkte auseinander [11].

Und wer kann morgen noch prüfen? → Die Fähigkeitsfalle

- Nutzer überschätzen ihre Fähigkeit, KI-Output zu prüfen [12]
- KI-Nutzung führt zu Cognitive Offloading und sinkendem kritischen Denken [13]
- → Deskilling: Erfahrene verlieren Fähigkeiten [14]
- → Never-Skilling: Einsteiger erlernen Grundlagen nicht [15]



Daten: Deskilling [14]; No-Skilling [16]

Teufelskreis **Deskilling**

Mehr KI-Assistenz

Cognitive Offloading

Deskilling

Never Skilling

[12], [13], [14], [15]

Fernandes et al.: *AI makes you smarter but none the wiser*, **Computers in Human Behavior**, 2026.

Gerlich, M.: *AI Tools in Society: Impacts on Cognitive Offloading and the Future of Critical Thinking*, **Societies**, 2025.

Budzyń et al.: *Endoscopist deskilling risk after exposure to artificial intelligence in colonoscopy*, **Lancet Gastroenterology & Hepatology**, 2025.

Ke et al.: *AI-induced never-skilling in medical education*, **Nature Medicine**, 2026.

Selbst wenn Menschen prüfen, müssen sie dazu in der Lage bleiben. Nutzende überschätzen ihre Fähigkeit, KI-Output zu prüfen [12], und die KI-Nutzung führt zu Cognitive Offloading und sinkendem kritischem Denken [13]. Erfahrene verlieren dadurch Fähigkeiten (Deskilling): In der Studie zur Koloskopie sank die Adenom-Erkennungsrate ohne KI-Hilfe von 28,4 auf 22,4 Prozent, nachdem die Ärztinnen und Ärzte KI genutzt hatten [14]. Einsteiger erlernen die Grundlagen gar nicht erst (Never-Skilling): Beim Code-Verständnis erreichten Lernende, die mit KI gearbeitet hatten, 50 statt 67 Prozent [15], [16]. So entsteht ein Teufelskreis: Mehr KI-Assistenz führt zu Cognitive Offloading, dieses zu Deskilling und am Ende zu Never-Skilling.

Was tun? → Den Teufelskreis durchbrechen

1. Bildung
- **Weiterbildung** für strukturierte KI-Nutzung
- **Ausbildung** für kritische Kompetenzen
- **Kultur** des Lernens und kritischen Denkens

2. Organisation
- **Prozesse** zur Prüfung von KI-Ergebnissen
- **Digitale Werkzeuge** zur Dokumentation im BIM
- **Führung**, die KI-Resilienz etabliert

3. Rahmenbedingungen
- **Forschung** zur Bewertung von KI-Resilienz
- **Regulierung** der Randbedingungen
- **Offenheit** für eine positive KI-Vision

Es braucht Zusammenarbeit, um Erfahrungen auszutauschen und Leitlinien zu entwickeln.

Den Teufelskreis durchbrechen wir auf drei Ebenen. In der Bildung geht es um Weiterbildung für strukturierte KI-Nutzung, Ausbildung für kritische Kompetenzen und eine Kultur des Lernens und kritischen Denkens. In der Organisation braucht es Prozesse zur Prüfung von KI-Ergebnissen, digitale Werkzeuge zur Dokumentation im BIM und Führung, die KI-Resilienz etabliert. Bei den Rahmenbedingungen gehören Forschung zur Bewertung von KI-Resilienz, Regulierung der Randbedingungen und Offenheit für eine positive KI-Vision dazu. Dafür braucht es Zusammenarbeit, um Erfahrungen auszutauschen und Leitlinien zu entwickeln.

Der Zyklus ist die Methode des ganzen Kurses

Block	Thema	Trainierter Schritt
1	Computational Foundations	Verstehen
2	Computational Engineering	Prompt
3	Programming Foundations	Verstehen, Iterieren
4	Verification & Quality	Verifizieren
5	Problem Classes	Prompt, Verifizieren

Block	Thema	Trainierter Schritt
6	Data Management	ganzer Zyklus

Jede Übung trainiert mindestens einen Schritt des Zyklus.

Der agentische Zyklus verteilt sich über alle sechs Kursblöcke (S01 bis S13). In Block 1 steht das Verstehen im Vordergrund: Code lesen, Datentypen erkennen. Block 2 trainiert die Formulierung von Anforderungen als Prompt. In Block 3 lesen und überarbeiten wir Kontrollstrukturen und Funktionen. Block 4 formalisiert die Verifikation durch Unit Tests und Code Review. In Block 5 wenden wir Prompt und Verifikation auf typische Problemklassen an, und Block 6 verbindet alle Schritte in datenbankgestützten Projekten. Die Zuordnung ist eine Orientierung, jede Sitzung kann weitere Schritte berühren.

Quiz

Referenzen

Bibliographie

[1] A. Karpathy, „There's a new kind of coding I call ``vibe coding''“. Februar 2025.

[2] A. Vaswani *u. a.*, „Attention is All You Need“, *NeurIPS*, Bd. 30, 2017, [Online]. Verfügbar unter: https://arxiv.org/abs/1706.03762

[3] L. Ouyang *u. a.*, „Training language models to follow instructions with human feedback“, *arXiv preprint arXiv:2203.02155*, 2022, [Online]. Verfügbar unter: https://arxiv.org/abs/2203.02155

[4] J. Ploennigs *u. a.*, „Building Foundation Models - Potentials, Challenges and Research Directions for Using LLM and LVM in AEC“, in *EC3*, 2025.

[5] V. Pursnani, Y. Sermet, M. Kurt, und I. Demir, „Performance of ChatGPT on the US fundamentals of engineering exam“, *Computers and Education: AI*, Bd. 5, S. 100183, 2023, doi: https://doi.org/10.1016/j.caeai.2023.100183.

[6] L. Oblitas, S. Adeeb, und C. Cruz-Noguez, „Performance of AI Chatbots on the Fundamentals of Engineering Civil Exam“, *Discover Artificial Intelligence*, Bd. 5, S. 378, 2025, doi: 10.1007/s44163-025-00646-6.

[7] T. B. Brown *u. a.*, „Language Models are Few-Shot Learners“, *NeurIPS*, Bd. 33, S. 1877–1901, 2020, [Online]. Verfügbar unter: https://arxiv.org/abs/2005.14165

[8] M. Chen, J. Tworek, H. Jun, Q. Yuan, und others, „Evaluating Large Language Models Trained on Code“, *arXiv preprint arXiv:2107.03374*, 2021, [Online]. Verfügbar unter: https://arxiv.org/abs/2107.03374

[9] K. North, *Wissensorientierte Unternehmensführung: Wertschöpfung durch Wissen*, 5. Aufl. Wiesbaden: Gabler, 2011.

[10] CodeRabbit, „GitHub Gives Maintainers a Throttle for the AI Pull Request“. Zugegriffen: 17. September 2026. [Online]. Verfügbar unter: https://www.coderabbit.ai/blog/github-gives-maintainers-a-throttle-for-the-ai-pull-request

[11] METR, „Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity“. [Online]. Verfügbar unter: https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

[12] D. Fernandes, T. Kosch, und P. W. Woźniak, „AI makes you smarter but none the wiser: The disconnect between performance and metacognition“, *Computers in Human Behavior*, 2026, doi: 10.1016/j.chb.2025.108779.

[13] M. Gerlich, „AI Tools in Society: Impacts on Cognitive Offloading and the Future of Critical Thinking“, *Societies*, Bd. 15, Nr. 1, S. 6, 2025, doi: 10.3390/soc15010006.

[14] K. Budzyń, M. Romańczyk, D. Kitala, und others, „Endoscopist deskilling risk after exposure to artificial intelligence in colonoscopy: a multicentre, observational study“, *The Lancet Gastroenterology & Hepatology*, Bd. 10, Nr. 10, S. 896–903, 2025, doi: 10.1016/S2468-1253(25)00133-5.

[15] Y. Ke, L. Jin, J. C. L. Ong, und others, „AI-induced never-skilling in medical education“, *Nature Medicine*, Bd. 32, Nr. 6, S. 1997–2006, 2026, doi: 10.1038/s41591-026-04438-y.

[16] J. H. Shen und A. Tamkin, „How AI Impacts Skill Formation“, *arXiv:2601.20245*, 2026.